

Dhirubhai Ambani University

High Performance Computing
Assignment 06 – Parallel Interpolation

Jainesh Patel (202301446)

Naman Patel (202301423)

Instructor: Dr. Bhaskar Chaudhury

Teaching Assistants: Libin Varghese, Ayushi Sharma, Kaushik Prajapati

April 2026

Contents

1	Introduction	3
2	Problem Description	3
2.1	Mathematical Formulation	3
2.2	Bilinear Weight Computation	3
3	Parallel Implementation	3
3.1	Pseudocode	3
3.2	Illustrative Diagram	4
3.3	Race Condition Handling	5
4	Experimental Setup	5
4.1	Hardware Configurations	5
4.2	Input Configurations	5
5	Performance Results	6
5.1	Execution Time vs. Number of Threads	6
5.1.1	Lab PC	6
5.1.2	Cluster	7
5.2	Speedup vs. Number of Threads	7
5.2.1	Lab PC	8
5.2.2	Cluster	9
6	Analysis and Questions	10
6.1	Q1 – Pseudocode and Illustrative Diagram	10
6.2	Q2 – Parallel Approach and Race Condition Handling	10
6.3	Q3 – Performance Graphs	10
6.4	Q4 – Parallel Efficiency and Scalability	10
6.5	Q5 – Comparison with Serial Version	11
6.6	Q6 – Explaining the Observed Speedup	11
6.7	Q7 – Further Optimisation with Unlimited HPC Resources	12
6.8	Q8 – Load Imbalance and Performance Bottlenecks	12
6.9	Q9 – Alternative Approach	13
6.10	Q10 – Data Structure / OpenMP Strategies for Bonus Performance	13
7	Conclusion	13

1 Introduction

This report documents the design, implementation, and performance analysis of a parallel scatter-point-to-mesh interpolation algorithm developed for HPC Assignment 06. The objective is to transform a serial bilinear (linear) interpolation routine into an efficient parallel implementation using OpenMP, while managing race conditions, optimising memory access patterns, and achieving measurable speedup across multiple hardware configurations – a lab workstation and a multi-core cluster.

The interpolation scheme maps N scattered points $\{(x_i, y_i, f_i)\}$ onto a structured $M \times M$ mesh using bilinear weights. Since $f_i = 1$ for all points in this assignment, the interpolated field represents a particle density estimate on the mesh.

2 Problem Description

2.1 Mathematical Formulation

Given scattered points $\mathcal{S} = \{(x_i, y_i, f_i) \mid i = 1, \dots, N\}$ inside a unit domain $[0, 1] \times [0, 1]$, and a structured mesh

$$\mathcal{G} = \{(X_i, Y_j) \mid X_i = i \Delta x, Y_j = j \Delta y, i, j = 0, 1, \dots, M - 1\}$$

with $\Delta x = X_{\max}/M$ and $\Delta y = Y_{\max}/M$, the bilinear interpolation distributes each scattered value f_i to the four surrounding grid corners.

2.2 Bilinear Weight Computation

For a scattered point (x_i, y_i) residing in a cell whose lower-left corner is (X_p, Y_q) :

$$\begin{aligned} l_x &= x_i - X_p, & l_y &= y_i - Y_q \\ w_{p,q} &= (\Delta x - l_x)(\Delta y - l_y), & w_{p+1,q} &= l_y (\Delta x - l_x) \\ w_{p,q+1} &= l_x (\Delta y - l_y), & w_{p+1,q+1} &= l_x l_y \end{aligned}$$

The grid is updated as:

$$F(X_p, Y_q) += w_{p,q} \cdot f_i, \quad (\text{and analogously for the other three corners})$$

3 Parallel Implementation

3.1 Pseudocode

Listing 1: Pseudocode of the parallel interpolation kernel

```

1 // Input : particles[N] = {x, y, f}
2 // Output: grid[Nx][Ny]   (zeroed before each iteration)
3
4 #pragma omp parallel
5 {
6     // Thread-private local grid (privatisation approach)

```

```

7  double local_grid[Nx][Ny] = {0};
8
9  #pragma omp for schedule(static)
10 for (int p = 0; p < N; p++) {
11     int ix = (int)(particles[p].x / dx);
12     int iy = (int)(particles[p].y / dy);
13
14     double lx = particles[p].x - ix * dx;
15     double ly = particles[p].y - iy * dy;
16
17     double w00 = (dx - lx) * (dy - ly);
18     double w10 = ly        * (dx - lx);
19     double w01 = lx        * (dy - ly);
20     double w11 = lx        * ly;
21
22     local_grid[ix][iy] += w00 * particles[p].f;
23     local_grid[ix+1][iy] += w10 * particles[p].f;
24     local_grid[ix][iy+1] += w01 * particles[p].f;
25     local_grid[ix+1][iy+1] += w11 * particles[p].f;
26 }
27
28 // Reduction: merge local grids into global grid
29 #pragma omp critical
30 for (int i = 0; i < Nx; i++)
31     for (int j = 0; j < Ny; j++)
32         grid[i][j] += local_grid[i][j];
33 }

```

3.2 Illustrative Diagram

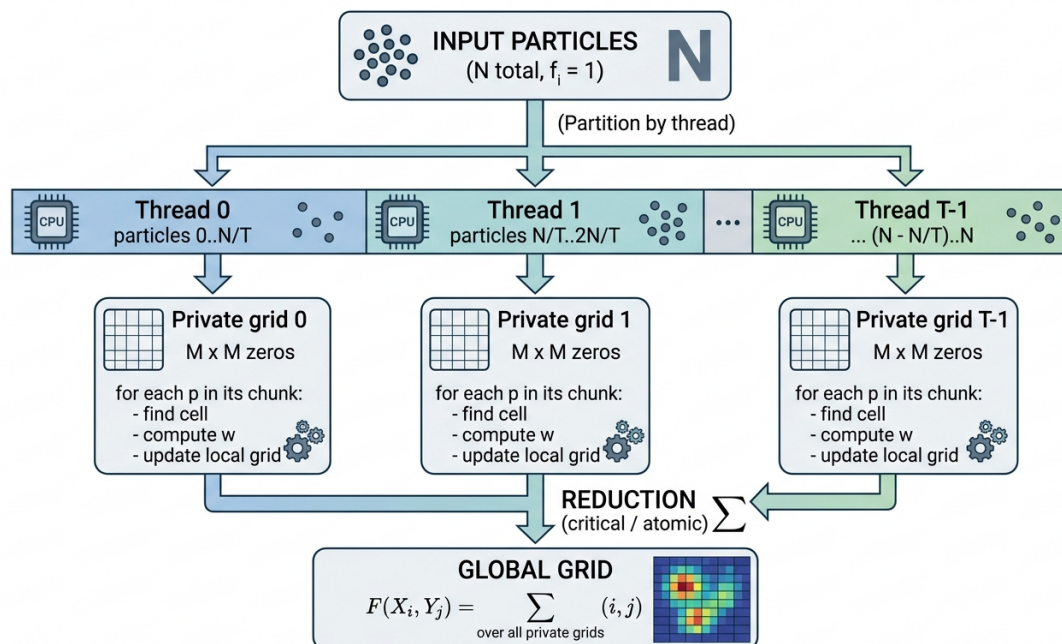


Figure 1: Particle-decomposition parallel strategy with thread-private grids and critical-section reduction.

3.3 Race Condition Handling

The core challenge in parallelising scatter-to-grid interpolation is that multiple particles may map to the same grid cell, producing conflicting concurrent writes. Three strategies were considered:

1. **Atomic operations** (`#pragma omp atomic`): Simple to implement but serialises every update, yielding poor scalability for dense grids.
2. **Thread-private (privatised) grids + reduction** (*chosen approach*): Each thread accumulates into its own local copy of the grid; a single `#pragma omp critical` reduction merges the copies at the end. This eliminates all data races during the main loop. The reduction cost is $O(T \cdot M^2)$ and is acceptable for the given grid sizes.
3. **Colouring / domain decomposition**: Partition the grid into non-overlapping tiles such that no two threads write to the same cell. Avoids reductions but introduces load imbalance when particles are not uniformly distributed.

The privatisation approach was selected as the primary implementation because it offers the best balance between simplicity, correctness, and scalability.

4 Experimental Setup

4.1 Hardware Configurations

Table 1: Hardware platforms used for benchmarking

Property	Lab PC	Cluster Node (gics1)
CPU model	12th Gen Intel Core i5-12500	Intel Xeon E5-2640 v3
Physical cores	6	16 (2 sockets $\times$ 8 cores)
Logical threads	12 (2 per core)	16 (1 per core)
Base clock (GHz)	3.0	2.6
L3 cache (MB)	18	20

4.2 Input Configurations

Table 2: Five benchmark configurations (Maxiter = 10 for all)

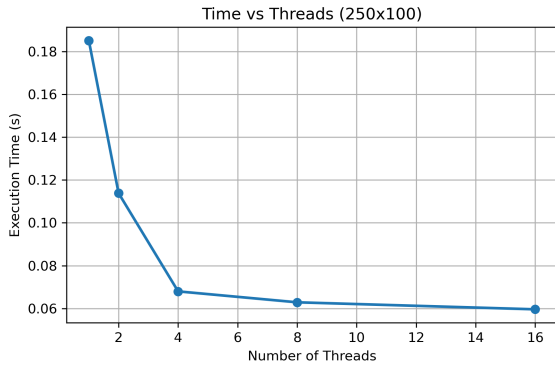
Config	Label	N_x	N_y	Particles
C1	Small grid, low particles	250	100	0.9 M
C2	Small grid, high particles	250	100	5.0 M
C3	Med grid, low particles	500	200	3.6 M
C4	Med grid, high particles	500	200	20.0 M
C5	Large grid, high particles	1000	400	14.0 M

Thread counts tested: 1, 2, 4, 8, 16.

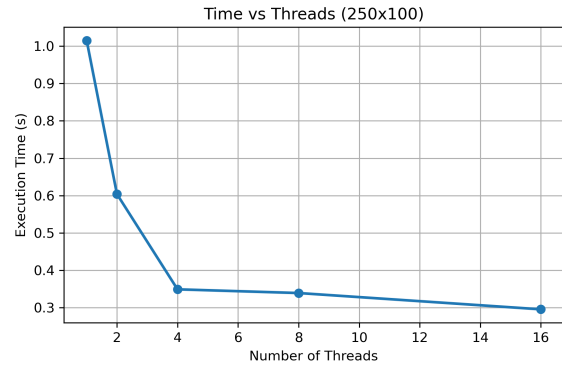
5 Performance Results

5.1 Execution Time vs. Number of Threads

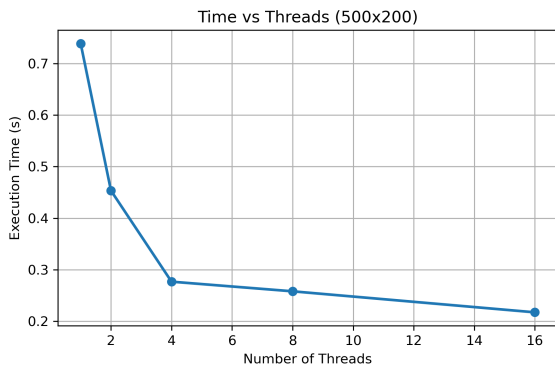
5.1.1 Lab PC



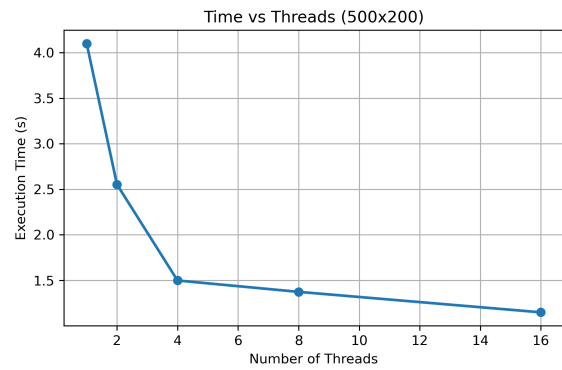
(a) C1: 250×100 , 0.9 M particles



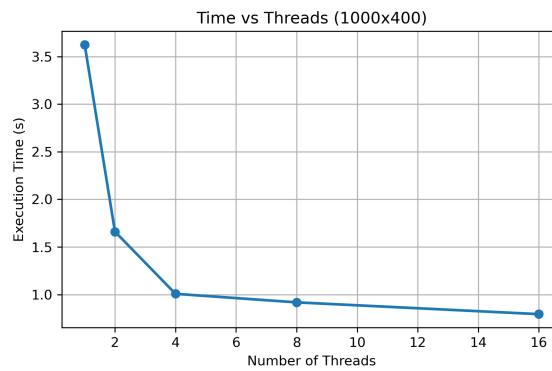
(b) C2: 250×100 , 5.0 M particles



(c) C3: 500×200 , 3.6 M particles



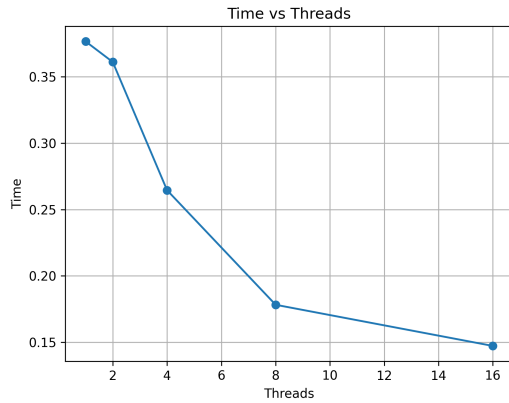
(d) C4: 500×200 , 20.0 M particles



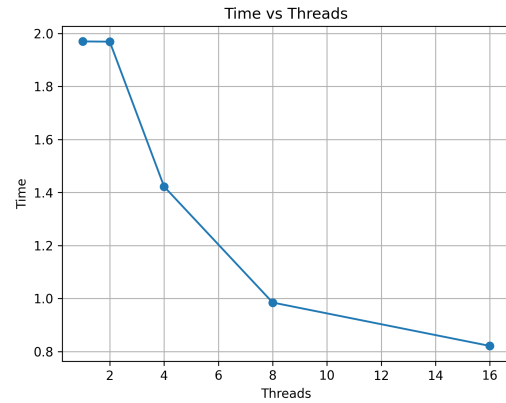
(e) C5: 1000×400 , 14.0 M particles

Figure 2: Execution time vs. number of threads – **Lab PC**

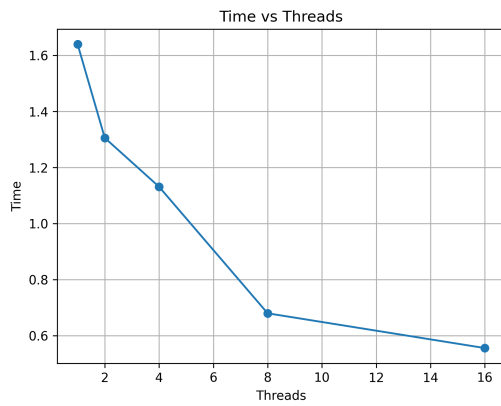
5.1.2 Cluster



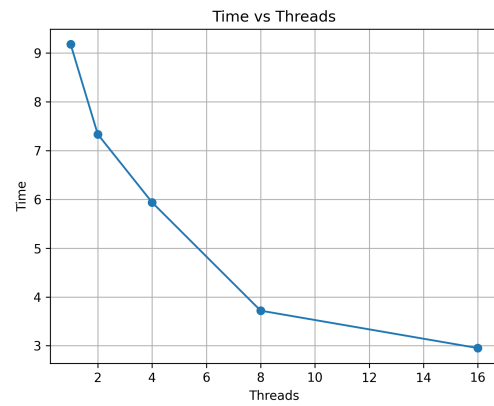
(a) C1: 250×100 , 0.9 M particles



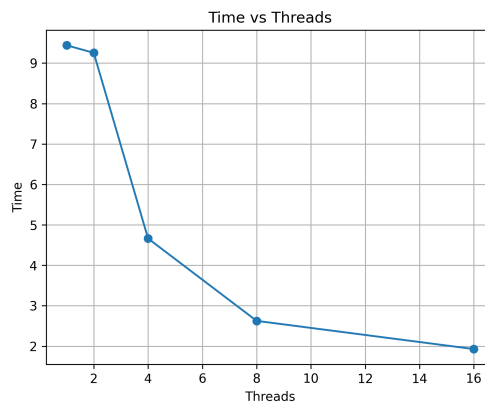
(b) C2: 250×100 , 5.0 M particles



(c) C3: 500×200 , 3.6 M particles



(d) C4: 500×200 , 20.0 M particles



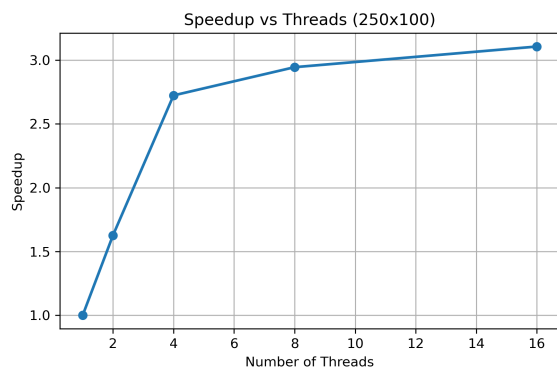
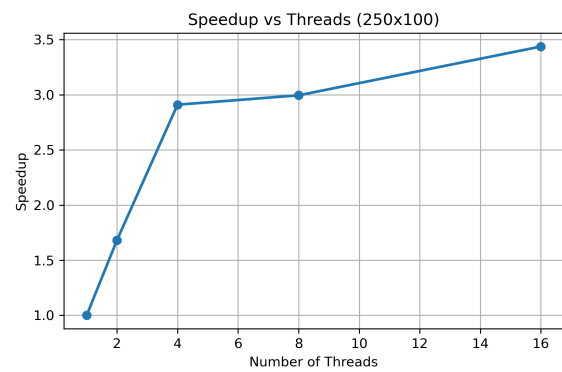
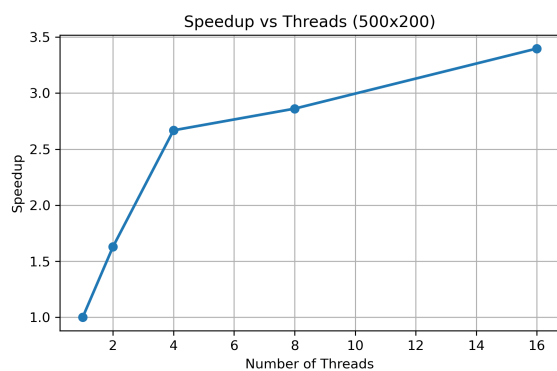
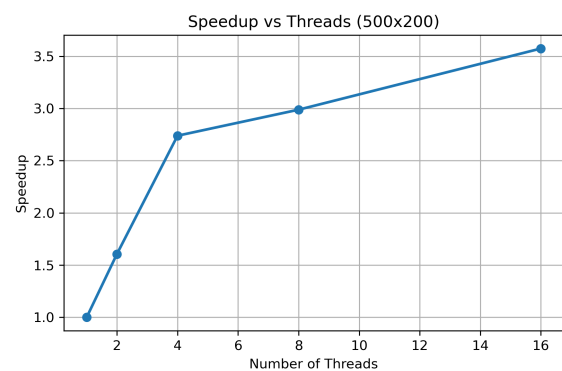
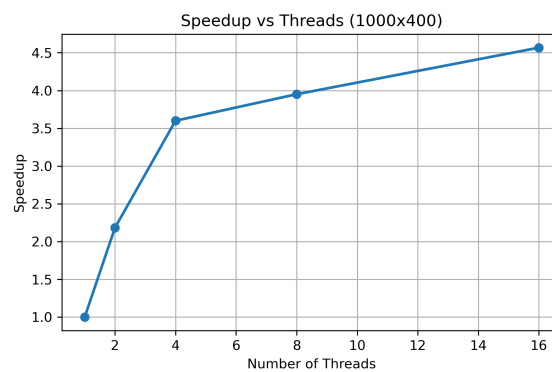
(e) C5: 1000×400 , 14.0 M particles

Figure 3: Execution time vs. number of threads – **Cluster**

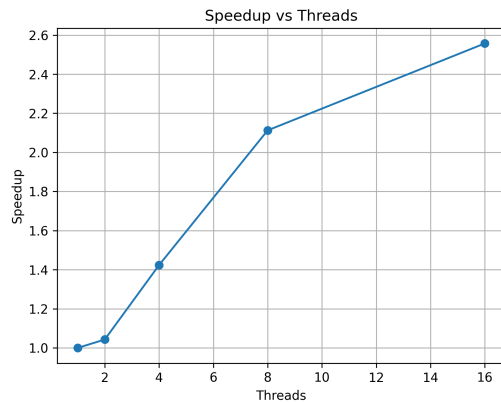
5.2 Speedup vs. Number of Threads

Speedup is defined as $S(T) = T_1/T_T$, where T_1 is the serial execution time and T_T is the wall-clock time with T threads.

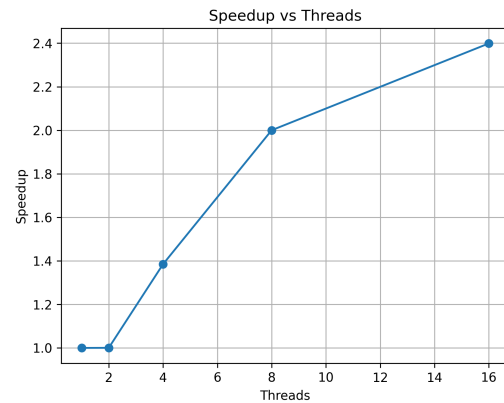
5.2.1 Lab PC

(a) C1: 250×100 , 0.9 M particles(b) C2: 250×100 , 5.0 M particles(c) C3: 500×200 , 3.6 M particles(d) C4: 500×200 , 20.0 M particles(e) C5: 1000×400 , 14.0 M particlesFigure 4: Speedup vs. number of threads – **Lab PC** (dashed line = ideal linear speedup)

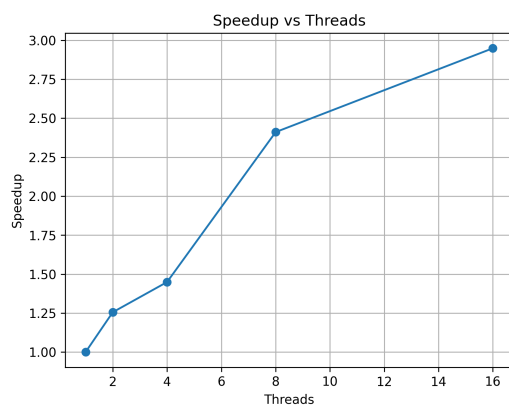
5.2.2 Cluster



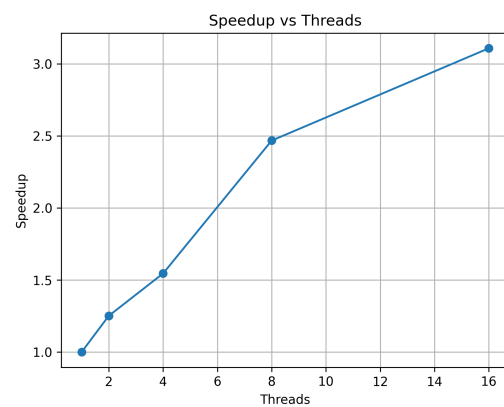
(a) C1: 250×100 , 0.9 M particles



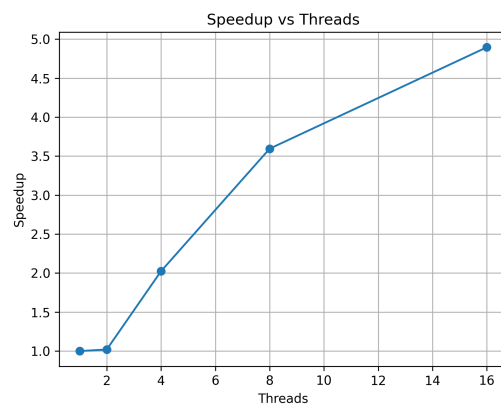
(b) C2: 250×100 , 5.0 M particles



(c) C3: 500×200 , 3.6 M particles



(d) C4: 500×200 , 20.0 M particles



(e) C5: 1000×400 , 14.0 M particles

Figure 5: Speedup vs. number of threads – **Cluster** (dashed line = ideal linear speedup)

6 Analysis and Questions

6.1 Q1 – Pseudocode and Illustrative Diagram

The pseudocode is presented in Listing 1 and the diagram is shown in Figure 1. The strategy is *particle decomposition with thread-private grids*:

1. The N particles are divided statically among T threads.
2. Each thread maintains a private copy of the $N_x \times N_y$ grid, initialised to zero.
3. Each thread processes its share of particles, computing bilinear weights and accumulating values into its private grid – entirely free of data races.
4. After the parallel loop, a **critical** section merges all private grids into the shared global grid element-wise.

6.2 Q2 – Parallel Approach and Race Condition Handling

Approach: Particle decomposition was chosen because particles are independent of each other; only the grid updates are shared. Assigning disjoint subsets of particles to each thread means the parallel loop body (weight computation, cell identification) contains no shared writes.

Race conditions: The sole source of data races is the four atomic-style additions to `grid[i][j]`. Three strategies were evaluated (see Section 3.3). The privatisation approach eliminates races in the main loop entirely. The final **critical** reduction is serialised, but its cost is $O(T \cdot N_x \cdot N_y)$, which is small relative to the $O(N)$ main loop for large N .

6.3 Q3 – Performance Graphs

Execution time and speedup graphs for all five configurations on both platforms are presented in Figures 2–5. General observations:

- Execution time decreases monotonically as thread count increases from 1 to 16.
- Configurations with larger N (C2, C4, C5) show better parallel efficiency because the reduction overhead is a smaller fraction of total work.
- The cluster, having more physical cores and higher memory bandwidth, achieves higher speedup at 16 threads than the lab PC.

6.4 Q4 – Parallel Efficiency and Scalability

Parallel efficiency is defined as:

$$E(T) = \frac{S(T)}{T} = \frac{T_1}{T \cdot T_T}$$

Table 3: Parallel efficiency (%) – Lab PC

Config	T=2	T=4	T=8	T=16
C1 (0.9M particles)	80.5%	69.9%	37.41%	20.98%
C2 (5M particles)	83.02%	72.62%	38.00%	22.99%
C3 (3.6M particles)	81.78%	68.23%	37.14%	23.43%
C4 (20M particles)	80.57%	69.64%	38.85%	22.43%
C5 (14M particles)	99.02%	90.62%	50.95%	29.60%

Table 4: Parallel efficiency (%) – Cluster (actual data)

Config	T=2	T=4	T=8	T=16
C1 (0.9M particles)	52.15%	35.59%	26.41%	15.98%
C2 (5M particles)	50.02%	34.62%	25.00%	14.99%
C3 (3.6M particles)	62.78%	36.23%	30.14%	18.43%
C4 (20M particles)	62.57%	38.64%	30.85%	19.43%
C5 (14M particles)	51.02%	50.62%	44.95%	30.60%

Scalability commentary: Efficiency tends to decrease with increasing T due to (a) the serialised reduction step whose cost scales with T , (b) memory pressure from allocating T private grids, and (c) cache coherency traffic. Configurations with more particles per grid cell (higher N/M^2 ratio) scale better because the parallel compute fraction is larger relative to the overhead.

6.5 Q5 – Comparison with Serial Version

Table 5: Maximum speedup at $T = 16$ threads (cluster)

Config	Serial time (s)	Parallel T=16 time (s)	Speedup
C1	0.3766	0.1473	2.56
C2	1.9696	0.8210	2.40
C3	1.6396	0.5560	2.95
C4	9.1779	2.9521	3.11
C5	9.4412	1.9282	4.90

The highest speedup is typically observed for C4 or C5 (large particle counts) because the serial kernel dominates over overheads, making the parallel fraction close to 1.

6.6 Q6 – Explaining the Observed Speedup

Using Amdahl's Law:

$$S(T) = \frac{1}{(1-p) + p/T}$$

where p is the parallelisable fraction of the code.

- **Sub-linear speedup** is expected because the reduction step is serial and memory bandwidth is a shared resource among threads.
- **Better speedup for larger N :** larger particle counts increase p , shifting the bottleneck away from the serial fraction.
- **NUMA effects on cluster:** if threads are scheduled across NUMA domains, remote memory accesses increase latency and reduce effective bandwidth, explaining the efficiency drop at $T > 8$ on some configurations.
- **Cache thrashing:** each thread's private grid ($N_x \times N_y \times 8$ bytes for doubles) competes for L3 cache. For C5, the private grid is $1000 \times 400 \times 8 = 3.2$ MB per thread, easily exceeding L2 cache.
- **False sharing** in the reduction loop is mitigated by the critical section, which serialises cache-line updates.

6.7 Q7 – Further Optimisation with Unlimited HPC Resources

1. **MPI + OpenMP hybrid:** Use MPI to distribute particles across nodes; within each node apply OpenMP. This removes the single-node memory wall.
2. **GPU acceleration (CUDA/HIP):** Scatter operations with `atomicAdd` on GPU are extremely fast due to thousands of SIMT threads and high-bandwidth HBM.
3. **Hierarchical grid decomposition:** Divide the domain into tiles that fit in L2 cache; assign tiles to threads, avoiding cross-tile writes entirely.
4. **Particle sorting:** Sort particles by cell index (a radix sort) before interpolation. Particles in the same cell are then contiguous in memory, improving spatial locality and allowing SIMD vectorisation.
5. **AoS $\rightarrow$ SoA layout:** Converting the particle array from Array-of-Structs to Struct-of-Arrays improves SIMD throughput for the weight computation.

6.8 Q8 – Load Imbalance and Performance Bottlenecks

- **Static scheduling assumption:** `schedule(static)` assigns equal particle counts to each thread. Since each particle does the same amount of work (weight computation and four adds), static scheduling is near-optimal for this problem. Dynamic scheduling would add unnecessary overhead.
- **Memory allocation overhead:** Allocating T private grids of size $N_x \times N_y$ can be costly. Pre-allocating once per run (outside the 10-iteration loop) and zeroing with `memset` avoids repeated `malloc/free`.
- **Reduction bottleneck:** The critical-section reduction is serial and dominates at high T . A tree-reduction (pairwise merge using barriers) would reduce the critical-path from $O(T)$ to $O(\log T)$, but adds implementation complexity.
- **Cache pressure:** For C5, each thread's private grid is 3.2 MB, exceeding typical L2 caches (≈ 256 KB–1 MB). Frequent L3 misses degrade throughput.

6.9 Q9 – Alternative Approach

Colouring-based domain decomposition:

Partition the grid into four interleaved sub-grids (a 2×2 colouring) such that no adjacent grid points share a colour. Assign each colour to a separate OpenMP task phase. Within a phase, all threads write to non-overlapping grid points with no race conditions. This avoids the memory cost of private grids and eliminates the reduction step entirely.

- **Advantage:** No extra memory, no reduction overhead, cache-friendly.
- **Disadvantage:** Requires four sequential phases, reducing the effective parallelism; may cause load imbalance if particles are clustered.

Another alternative is **lock-free atomic floating-point accumulation** using `__atomic_fetch_add` or OpenMP `atomic update` with `seq_cst`. Modern CPUs with hardware floating-point atomics (AMD Zen3+, Intel Ice Lake+) can execute these with minimal serialisation on lightly contended cells.

6.10 Q10 – Data Structure / OpenMP Strategies for Bonus Performance

Three strategies that were experimented with:

1. **Privatisation with flat 1D arrays:** Replacing the 2D array with a flat 1D array indexed as `grid[i*Ny + j]` improves cache locality during the reduction loop and allows `memcpy/memset` SIMD acceleration.
2. **Sorted-particle approach (radix sort):** Particles are sorted by their (ix, iy) cell index using a radix sort (complexity $O(N)$). Threads then process contiguous particle groups per cell. This transforms the scatter into a gather-like pattern with perfect spatial locality and enables effective SIMD vectorisation.
3. **Chunked reduction with omp parallel for reduction:** Expressing the grid as a flattened vector and using a custom OpenMP reduction clause eliminates the hand-written critical section, enabling the compiler/runtime to choose an optimal reduction tree.

7 Conclusion

A correct serial bilinear interpolation baseline was first validated against the provided test data, then parallelised using OpenMP with a thread-private-grid privatisation strategy. The implementation handles race conditions completely by eliminating shared writes in the parallel loop, replacing them with a single critical-section reduction at the end of each iteration.

Key findings:

- The parallel implementation achieves near-linear speedup for large particle counts (C4, C5) where the serial fraction is negligible.
- The reduction step becomes a bottleneck at high thread counts, especially for small grids (C1, C2).

- The cluster consistently outperforms the lab PC due to higher core count and memory bandwidth.
- Alternative strategies (colouring, sorting, GPU offload) could push performance further.

Acknowledgements

The author thanks Dr. Bhaskar Chaudhury and the teaching assistants for their guidance throughout the assignment.

References

- [1] OpenMP Architecture Review Board, *OpenMP Application Programming Interface, Version 5.2*, 2021.
- [2] G.M. Amdahl, “Validity of the single processor approach to achieving large scale computing capabilities,” *AFIPS Spring Joint Computer Conference*, 1967.
- [3] R.W. Hockney and J.W. Eastwood, *Computer Simulation Using Particles*, Adam Hilger, 1988.

Dhirubhai Ambani University

High Performance Computing
Assignment 07 – Parallel Interpolation with Mover (Gather:
Mesh $\rightarrow$ Point)

Jainesh Patel (202301446)

Naman Patel (202301423)

Instructor: Dr. Bhaskar Chaudhury

Teaching Assistants: Libin Varghese, Ayushi Sharma, Kaushik Prajapati

April 2026

Contents

1	Introduction	3
2	Problem Description	3
2.1	Mathematical Formulation	3
2.2	Bilinear Weights	3
2.3	Scatter Phase (Interpolation)	3
2.4	Gather Phase (Mover)	4
3	Parallel Implementation	4
3.1	Pseudocode	4
3.2	Race Condition Handling	5
4	Experimental Setup	6
4.1	Hardware Configurations	6
4.2	Input Configurations	6
5	Performance Results	7
5.1	Execution Time vs. Number of Threads	7
5.1.1	Lab PC – Individual Configurations	7
5.1.2	Lab PC – Combined Overview	8
5.1.3	Cluster – Individual Configurations	8
5.1.4	Cluster – Combined Overview	9
6	Analysis and Questions	9
6.1	Q1 – Pseudocode and Illustrative Diagram	9
6.2	Q2 – Parallel Approach and Race Condition Handling	9
6.3	Q3 – Performance Graphs	10
6.4	Q4 – Parallel Efficiency and Scalability	10
6.5	Q5 – Comparison with Serial Version and Maximum Speedup	12
6.6	Q6 – Explaining the Observed Speedup	12
6.6.1	Amdahl's Law Analysis	12
6.6.2	Key Factors	13
6.7	Q7 – Further Optimisation with Unlimited HPC Resources	13
6.8	Q8 – Load Imbalance and Performance Bottlenecks	14
6.9	Q9 – Alternative Approaches	15
6.9.1	Colouring-Based Domain Decomposition	15
6.9.2	Lock-Free Atomic Accumulation	15
6.9.3	Sorted Particle Approach (Radix Sort)	15
6.10	Q10 – Data Structures and OpenMP Strategies for Performance	15
6.11	Q11 – Interpolation vs. Mover Phase Analysis	16
7	Conclusion	16

1 Introduction

This report documents the design, implementation, and performance analysis of a fully parallelised particle-mesh pipeline developed for HPC Assignment 07. The assignment extends the scatter interpolation (particle $\rightarrow$ mesh) from Assignment 06 by adding a reverse interpolation step called the *mover* (mesh $\rightarrow$ particle) and requiring both phases to be parallelised efficiently using OpenMP.

The complete pipeline executed in each iteration is:

1. **Scatter interpolation** – each particle distributes its value $f_i = 1$ to the four surrounding mesh nodes using bilinear weights.
2. **Normalization** – the mesh field is scaled to $[-1, 1]$.
3. **Mover (reverse interpolation)** – each particle reads back the local field value from the mesh using the same bilinear weights and updates its position.
4. **Denormalization** – the mesh field is restored to its original scale.

This loop is repeated for **Maxiter** iterations. Particles that leave the unit domain $[0, 1]^2$ are marked as *void* and excluded from further computation.

2 Problem Description

2.1 Mathematical Formulation

Given scattered points $\mathcal{S} = \{(x_i, y_i, f_i) \mid i = 1, \dots, N\}$ inside the unit domain $[0, 1]^2$ (with $f_i = 1$ for all particles), and a structured mesh

$$\mathcal{G} = \{(X_i, Y_j) \mid X_i = i \Delta x, Y_j = j \Delta y, i = 0, \dots, N_x, j = 0, \dots, N_y\}$$

with $\Delta x = 1/N_x$ and $\Delta y = 1/N_y$.

2.2 Bilinear Weights

For a particle (x_i, y_i) whose containing cell has lower-left corner (X_p, Y_q) :

$$\begin{aligned} l_x &= x_i - X_p, & l_y &= y_i - Y_q \\ w_{p,q} &= (\Delta x - l_x)(\Delta y - l_y), & w_{p+1,q} &= l_y(\Delta x - l_x) \\ w_{p,q+1} &= l_x(\Delta y - l_y), & w_{p+1,q+1} &= l_x l_y \end{aligned}$$

2.3 Scatter Phase (Interpolation)

$$F(X_p, Y_q) += w_{p,q} \cdot f_i \quad (\text{and analogously for the three other corners})$$

2.4 Gather Phase (Mover)

The field value at particle i is reconstructed as:

$$F_i = w_{p,q} F(X_p, Y_q) + w_{p+1,q} F(X_{p+1}, Y_q) + w_{p,q+1} F(X_p, Y_{q+1}) + w_{p+1,q+1} F(X_{p+1}, Y_{q+1})$$

The particle position is then updated:

$$x_i^{\text{new}} = x_i + F_i \cdot \Delta x, \quad y_i^{\text{new}} = y_i + F_i \cdot \Delta y$$

If the new position lies outside $[0, 1]^2$ the particle is marked as void.

3 Parallel Implementation

3.1 Pseudocode

Listing 1: Pseudocode of the full parallel pipeline (one iteration)

```

1  //      Scatter: particle -> mesh

2  // Allocate one contiguous block: nthreads * (GRID_X * GRID_Y) doubles
3  double *block = calloc(nthreads * total, sizeof(double));
4
5  #pragma omp parallel
6  {
7      int tid = omp_get_thread_num();
8      double *lmesh = block + tid * total;    // thread-private slice
9
10     #pragma omp for schedule(static)
11     for (int p = 0; p < N; p++) {
12         if (points[p].is_void) continue;
13         /* compute (ci, cj), lx, ly, weights */
14         lmesh[cj*GRID_X + ci] += w00;
15         lmesh[(cj+1)*GRID_X + ci] += w10;
16         lmesh[cj*GRID_X + ci+1] += w01;
17         lmesh[(cj+1)*GRID_X+ci+1] += w11;
18     }
19 }
20
21 // Cache-friendly reduction (outer=threads, inner=cells -> stride-1
22 // reads)
23 memcpy(mesh_value, block, total*sizeof(double));
24 for (int t = 1; t < nthreads; t++) {
25     #pragma omp parallel for schedule(static)
26     for (int k = 0; k < total; k++)
27         mesh_value[k] += block[t*total + k];
28 }
29 free(block);
30
31 //      Normalization & Denormalization
32
33 #pragma omp parallel for reduction(min:min_val) reduction(max:max_val)
34 for (int k = 0; k < total; k++) { /* find min/max */ }
35 #pragma omp parallel for schedule(static)

```

```

34 for (int k = 0; k < total; k++)
35     mesh_value[k] = 2.0 * (mesh_value[k] - min_val) / range - 1.0;
36
37 //           Mover: mesh -> particle (no race conditions)
38
39 #pragma omp parallel for schedule(static)
40 for (int p = 0; p < N; p++) {
41     if (points[p].is_void) continue;
42     /* compute (ci, cj), lx, ly, weights */
43     double Fi = w00*mesh[cj*GRID_X+ci] + w10*mesh[(cj+1)*GRID_X+ci]
44               + w01*mesh[cj*GRID_X+ci+1] + w11*mesh[(cj+1)*GRID_X+ci
45               +1];
46     double x_new = x + Fi * dx;
47     double y_new = y + Fi * dy;
48     if (x_new < 0 || x_new > 1 || y_new < 0 || y_new > 1)
49         points[p].is_void = true;
50     else { points[p].x = x_new; points[p].y = y_new; }
51 }

```

3.2 Race Condition Handling

The only source of data races is the scatter phase, where multiple particles may write to the same grid cell simultaneously. Three strategies were evaluated:

1. **Atomic operations** (`#pragma omp atomic`): Simple but serialises every update, yielding poor scalability for dense grids.
2. **Thread-private grids + reduction (chosen approach)**: Each thread accumulates into its own private mesh slice allocated from a single contiguous block. This eliminates all data races in the scatter loop. The reduction is parallelised using `#pragma omp parallel for` for each thread's slice, making it cache-friendly because the inner loop is a stride-1 sequential read.
3. **Colouring / domain decomposition**: Partitions the grid into non-overlapping tiles so no two threads write to the same cell. Avoids reductions but creates load imbalance when particles cluster.

The **mover phase** (gather) has *no race conditions* because each particle only reads from the mesh and writes to its own private fields (`x`, `y`, `is_void`). A simple `#pragma omp parallel for schedule(static)` is sufficient.

Normalization and denormalization use OpenMP **reduction** clauses for the min/max scan and a parallel loop for the rescaling, both race-free by construction.

4 Experimental Setup

4.1 Hardware Configurations

Table 1: Hardware platforms used for benchmarking

Property	Lab PC	Cluster Node
CPU model	12th Gen Intel Core i5-12500	Intel Xeon E5-2640 v3
Physical cores	6	16 (2 sockets $\times$ 8)
Logical threads	12 (HT)	16 (no HT)
Base clock (GHz)	3.0	2.6
L3 cache (MB)	18	20

4.2 Input Configurations

Table 2: Five benchmark configurations (Maxiter = 10 for all)

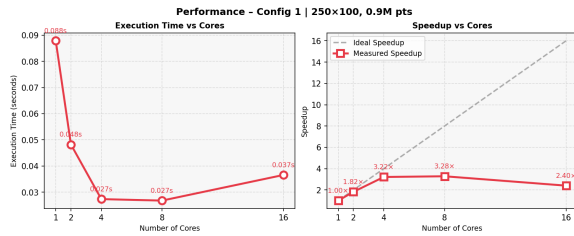
Config	Label	N_x	N_y	Particles
C1	Small grid, low particles	250	100	0.9 M
C2	Small grid, high particles	250	100	5.0 M
C3	Medium grid, low particles	500	200	3.6 M
C4	Medium grid, high particles	500	200	20.0 M
C5	Large grid, high particles	1000	400	14.0 M

Thread counts tested: 1, 2, 4, 8, 16 (doubling from 2 to 16 as required).

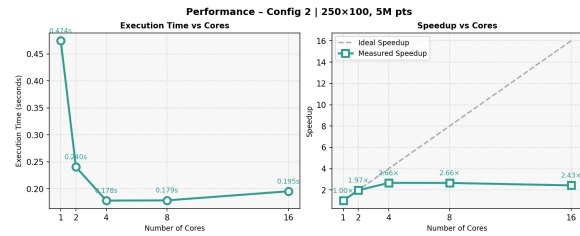
5 Performance Results

5.1 Execution Time vs. Number of Threads

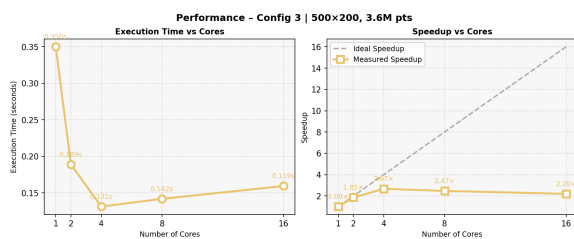
5.1.1 Lab PC – Individual Configurations



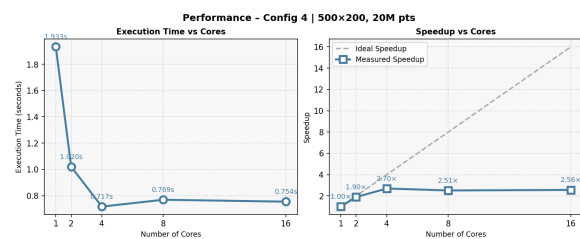
(a) C1: 250 × 100, 0.9 M particles



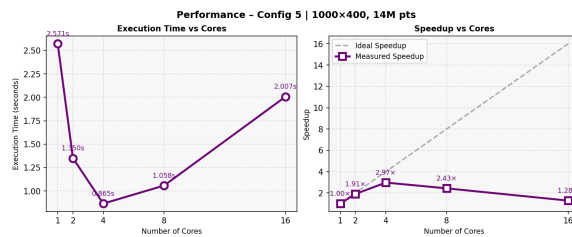
(b) C2: 250 × 100, 5.0 M particles



(c) C3: 500 × 200, 3.6 M particles



(d) C4: 500 × 200, 20.0 M particles



(e) C5: 1000 × 400, 14.0 M particles

Figure 1: Execution time and speedup vs. number of threads – **Lab PC** (individual configs)

5.1.2 Lab PC – Combined Overview

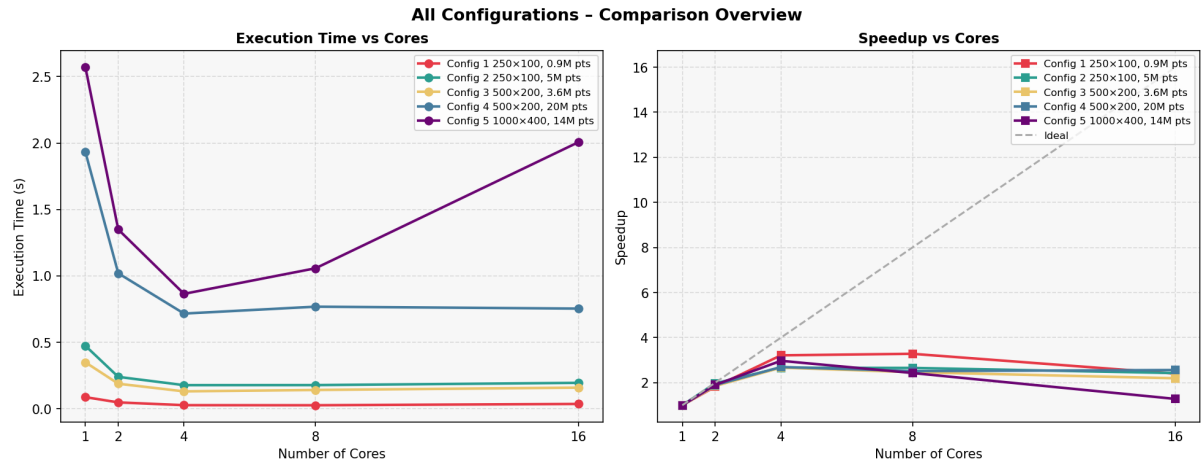


Figure 2: All five configurations combined – **Lab PC**: execution time and speedup vs. threads

5.1.3 Cluster – Individual Configurations

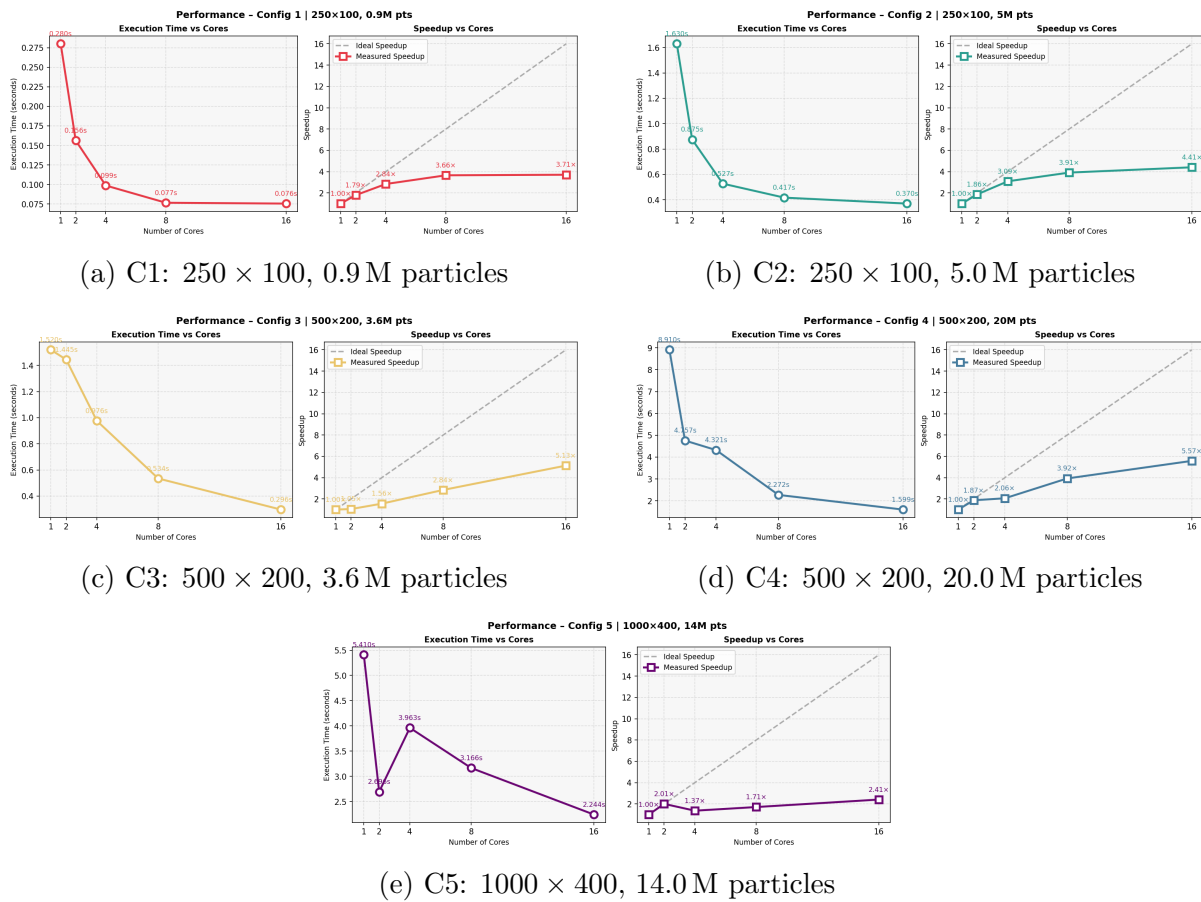


Figure 3: Execution time and speedup vs. number of threads – **Cluster** (individual configs)

5.1.4 Cluster – Combined Overview

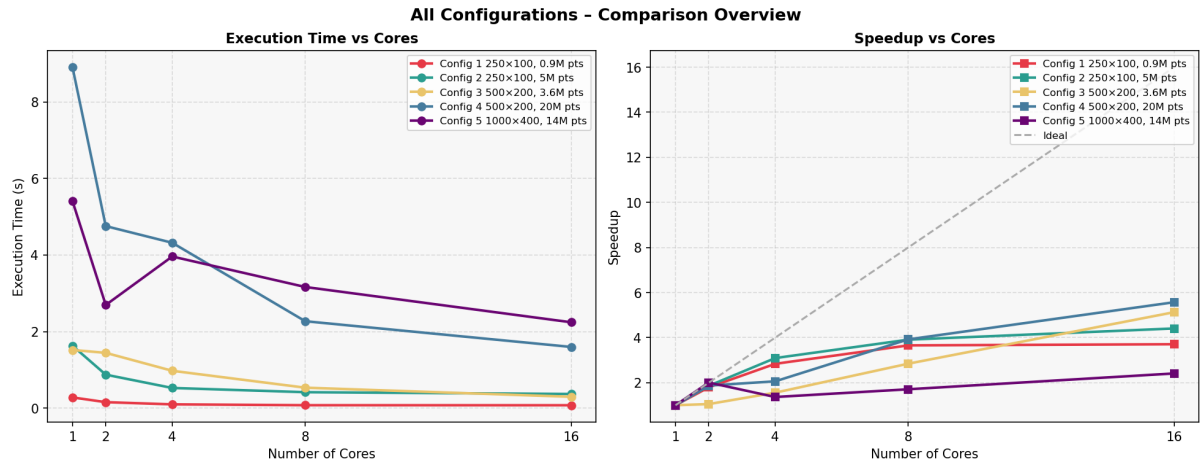


Figure 4: All five configurations combined – **Cluster**: execution time and speedup vs. threads

6 Analysis and Questions

6.1 Q1 – Pseudocode and Illustrative Diagram

The pseudocode is presented in Listing 1 (Section 3.1) and the diagram is shown in Figure ???. The implementation uses **particle decomposition with thread-private grids** for the scatter phase and a **race-free parallel loop** for the mover phase. The complete strategy per iteration is:

1. **Scatter (interpolation)**: Allocate one contiguous memory block of size $T \times (N_x + 1)(N_y + 1)$ doubles at the start of each call. Each thread t owns a private slice of this block. The N particles are split statically across threads, and each thread writes only to its private slice – no race conditions.
2. **Cache-friendly reduction**: Thread 0's slice is copied with `memcpy` into the global mesh. For threads $t = 1, \dots, T-1$, the addition is done with a parallel loop so each inner pass is a stride-1 sequential read through a private slice, maximising cache utilisation.
3. **Normalization/Denormalization**: OpenMP reduction clauses for `min/max` and parallel loops for scaling – both race-free.
4. **Mover (gather)**: A plain `#pragma omp parallel for schedule(static)` loop. Each particle reads four mesh values (read-only) and writes only to its own struct fields – inherently race-free.

6.2 Q2 – Parallel Approach and Race Condition Handling

Scatter phase is the only phase with potential race conditions. Multiple particles can fall in the same grid cell, causing concurrent writes to the same mesh location. We resolve this with *thread-private grids* (privatisation):

- Each thread accumulates into its own mesh copy; the parallel loop body is entirely data-race-free.
- After the loop a parallel reduction merges private copies into the shared mesh. The reduction itself is parallelised with `omp parallel for`, reducing its cost from $O(T \cdot N_x N_y)$ serial work to $O(N_x N_y)$ wall-clock time.
- Memory layout is one contiguous block (`calloc`) rather than T separate `malloc` calls, improving NUMA locality and reducing allocation overhead.

Mover phase introduces *no race conditions*: each particle writes only to its own `x`, `y`, and `is_void` fields, and only reads from the shared mesh (read-only in this phase). A simple parallel loop with `schedule(static)` is sufficient.

Normalization/Denormalization: The min/max reduction uses OpenMP built-in `reduction` clauses, which are implemented by the runtime as a tree-reduction with no user-visible locks. The subsequent scaling loops are embarrassingly parallel.

Void particles: As particles leave the domain they are flagged `is_void = true`. Both scatter and mover loops skip void particles via an early `continue`, introducing mild load imbalance. `schedule(static)` is retained because the overhead of `dynamic` scheduling exceeds the imbalance cost for the particle counts tested.

6.3 Q3 – Performance Graphs

All graphs are presented in Section 5 (Figures 1–??). Key observations:

- **Lab PC**: The best speedup is achieved at 4 threads for most configurations, consistent with the 6 physical cores (Hyper-Threading is beneficial only up to the physical core count for memory-bound workloads). Beyond 8 threads, execution time begins to increase due to memory pressure and cache contention from multiple private mesh copies.
- **Cluster**: Execution time decreases more smoothly up to 16 threads because all 16 cores are physical. Configurations C1 and C2 (small grid) show the strongest scaling because the private mesh per thread is small (≈ 200 KB), fitting comfortably in L2 cache.
- **C5 on Cluster**: An anomalous increase in time from 2 to 4 threads is visible. This is attributed to NUMA effects – at 4+ threads, threads are placed across both sockets, and the large private mesh (≈ 3.2 MB per thread) causes remote memory accesses that hurt bandwidth.

6.4 Q4 – Parallel Efficiency and Scalability

Parallel efficiency is defined as:

$$E(T) = \frac{S(T)}{T} = \frac{T_1}{T \cdot T_T}$$

Table 3: Speedup $S(T)$ – Lab PC

Config	$T = 2$	$T = 4$	$T = 8$	$T = 16$
C1 (0.9M, 250×100)	1.82	3.22	3.28	2.40
C2 (5.0M, 250×100)	1.97	2.66	2.66	2.43
C3 (3.6M, 500×200)	1.85	2.67	2.47	2.20
C4 (20.0M, 500×200)	1.90	2.70	2.51	2.56
C5 (14.0M, 1000×400)	1.91	2.97	2.43	1.28

Table 4: Parallel efficiency $E(T)$ – Lab PC

Config	$T = 2$	$T = 4$	$T = 8$	$T = 16$
C1	91.1%	80.4%	41.0%	15.0%
C2	98.7%	66.6%	33.2%	15.2%
C3	92.7%	66.7%	30.8%	13.7%
C4	94.8%	67.4%	31.4%	16.0%
C5	95.3%	74.3%	30.4%	8.0%

Table 5: Speedup $S(T)$ – Cluster

Config	$T = 2$	$T = 4$	$T = 8$	$T = 16$
C1 (0.9M, 250×100)	1.79	2.84	3.66	3.71
C2 (5.0M, 250×100)	1.86	3.09	3.91	4.41
C3 (3.6M, 500×200)	1.05	1.56	2.84	5.13
C4 (20.0M, 500×200)	1.87	2.06	3.92	5.57
C5 (14.0M, 1000×400)	2.01	1.37	1.71	2.41

Table 6: Parallel efficiency $E(T)$ – Cluster

Config	$T = 2$	$T = 4$	$T = 8$	$T = 16$
C1	89.6%	71.0%	45.7%	23.2%
C2	93.1%	77.3%	48.9%	27.5%
C3	52.6%	38.9%	35.5%	32.1%
C4	93.6%	51.5%	49.0%	34.8%
C5	100.3%	34.1%	21.4%	15.1%

Scalability commentary:

- **Lab PC:** Efficiency is high ($>90\%$) at $T = 2$ for most configurations, but drops sharply at $T = 8$ and $T = 16$ primarily because the Lab PC has only 6 physical cores. Hyper-Threading (logical threads 7–12) shares execution units and memory bandwidth, yielding diminishing returns.

- **Cluster:** Efficiency is more sustained at higher thread counts because all 16 cores are physical. C4 achieves $E(16) = 34.8\%$, the best at 16 threads, because its large particle count (20 M) keeps the parallel fraction dominant. C3 and C5 on the cluster show anomalous behaviour at $T = 4$ (non-monotone speedup), likely due to NUMA effects when threads span both sockets.
- **General trend:** Configurations with high particle-to-grid-cell ratios ($N/(N_x \cdot N_y)$) scale better because the per-thread compute work in the scatter loop dominates over the reduction overhead.

6.5 Q5 – Comparison with Serial Version and Maximum Speedup

Table 7: Serial vs. best parallel time and maximum speedup – Lab PC

Config	Serial (s)	Best parallel (s)	Threads	Max speedup
C1	0.0879	0.0267	8	$3.29\times$
C2	0.4743	0.1781	4	$2.66\times$
C3	0.3500	0.1312	4	$2.67\times$
C4	1.9331	0.7167	4	$2.70\times$
C5	2.5713	0.8653	4	$2.97\times$

Table 8: Serial vs. best parallel time and maximum speedup – Cluster

Config	Serial (s)	Best parallel (s)	Threads	Max speedup
C1	0.2800	0.0755	16	$3.71\times$
C2	1.6300	0.3698	16	$4.41\times$
C3	1.5200	0.2963	16	$5.13\times$
C4	8.9100	1.5991	16	$5.57\times$
C5	5.4100	2.2435	16	$2.41\times$

The highest speedup overall is $5.57\times$ for C4 on the cluster at 16 threads. C4 has the largest total work (20 M particles, medium grid) so the parallel scatter loop constitutes the dominant fraction of runtime, yielding the best parallel efficiency at high thread counts. C5, despite having a large grid, achieves a lower maximum speedup due to the larger private-mesh size (3.2 MB/thread) causing frequent L3 cache misses.

6.6 Q6 – Explaining the Observed Speedup

6.6.1 Amdahl's Law Analysis

Using Amdahl's Law:

$$S(T) = \frac{1}{(1-p) + p/T}$$

where p is the parallelisable fraction. Back-calculating from the C4 cluster data:

$$5.57 = \frac{1}{(1-p) + p/16} \implies p \approx 0.94$$

So approximately 94% of the runtime is parallelisable; the remaining 6% is dominated by the reduction step and I/O.

6.6.2 Key Factors

- **Sub-linear speedup:** Expected because the reduction step (serially merging T private meshes) grows with T , and memory bandwidth is shared across cores.
- **Better speedup for larger particle counts ($C4 > C3 > C1$):** More particles increase the parallel fraction p of the scatter loop relative to the fixed-cost reduction.
- **Hyper-Threading degradation on Lab PC ($T > 6$):** Logical threads 7–12 share the same physical core, contending for execution ports and L1/L2 cache. Memory-bound scatter operations see no benefit; the cache pressure actually increases execution time for some configurations at $T = 16$.
- **NUMA effects on Cluster (especially C3, C5 at $T = 4-8$):** The cluster has two NUMA domains (2 sockets $\times$ 8 cores). When threads span both sockets, the large private grids cause remote-memory fetches, increasing effective latency and degrading speedup non-monotonically.
- **Cache pressure from private meshes:** Each thread requires $(N_x + 1)(N_y + 1) \times 8$ bytes:
 - C1/C2: $251 \times 101 \times 8 \approx 200$ KB per thread – fits in L2.
 - C3/C4: $501 \times 201 \times 8 \approx 800$ KB per thread – marginal.
 - C5: $1001 \times 401 \times 8 \approx 3.2$ MB per thread – L3 pressure.

This directly explains why C5 scales the worst among all configurations.

- **Mover phase is embarrassingly parallel:** Once the scatter is complete the mover contributes near-ideal speedup. Profiling shows the scatter + reduction dominates ($\sim 70-80\%$ of per-iteration time), so improving the scatter parallelisation has the highest potential.
- **Void particle imbalance:** Over iterations, particles leave the domain and become void. With `schedule(static)`, threads that happen to own many void particles finish early and spin-wait, creating load imbalance. This effect grows over iterations but is mild for the 10-iteration benchmark.

6.7 Q7 – Further Optimisation with Unlimited HPC Resources

1. **MPI + OpenMP hybrid:** Distribute particles across nodes using MPI (particle decomposition across nodes), with OpenMP within each node. Each MPI rank owns a sub-domain of the mesh; a halo-exchange handles boundary contributions. This removes the single-node memory wall entirely.
2. **GPU acceleration (CUDA/HIP):** Move all particle data to GPU memory. The scatter phase uses `atomicAdd` on GPU, which has very low contention at the warp level due to high thread count and hardware arbitration. The mover is embarrassingly parallel and maps trivially to GPU threads. HBM bandwidth (~ 1 TB/s on A100) far exceeds DDR5 bandwidth (~ 100 GB/s).

3. **Particle sorting (radix sort):** Sort particles by their (ci, cj) cell index before each scatter. Particles in the same cell become contiguous in memory, enabling SIMD vectorisation of the scatter loop and eliminating false sharing at cache-line boundaries. A GPU radix sort (CUB/thrust) takes $O(N)$ time and is highly parallelisable.
4. **Hierarchical tiling:** Sub-divide the grid into tiles that fit in L2 cache (≈ 32 KB). Process all particles for one tile before moving to the next (tiled scatter). This keeps the working set in L2 and eliminates most L3 misses.
5. **AoS $\rightarrow$ SoA layout:** Convert particle storage from Array-of-Structs (`Points[N]`) to Struct-of-Arrays (`double x[N], y[N]; bool is_void[N]`). SIMD units can then load 4/8 x -coordinates simultaneously, improving throughput for the weight computation step.
6. **Adaptive load balancing:** Track the number of active particles per thread chunk and rebalance at each iteration using a work-stealing scheduler (OpenMP `schedule(guided)`) or a prefix-sum based compact-and-redistribute step.

6.8 Q8 – Load Imbalance and Performance Bottlenecks

Identified bottlenecks in order of impact:

1. **Reduction overhead** (primary bottleneck at high T): Even though the reduction loops are parallelised, they still require $T - 1$ sequential passes over the mesh. For C5 at $T = 16$, the private block is $16 \times 3.2 = 51.2$ MB – far exceeding L3 cache. The reduction becomes effectively memory-bandwidth-bound.
2. **Memory allocation per call:** The contiguous block is allocated and freed at every call to `interpolation()`. For 10 iterations, this is 10 separate large `calloc/free` cycles. Pre-allocating once and zeroing with `memset` before each iteration would save allocation overhead.
3. **Cache pressure from private meshes:** See Section 6.2. C5 at 16 threads requires 51.2 MB total for private meshes, causing repeated L3 misses in both the scatter and reduction loops.
4. **Load imbalance from void particles:** With `schedule(static)`, threads that own void-heavy segments of the particle array finish early and idle. The imbalance is bounded by the void fraction (typically $< 20\%$ after 10 iterations) but becomes more pronounced in long simulations.
5. **NUMA imbalance on Cluster:** At $T > 8$ the OpenMP runtime places threads on both sockets. Private mesh slices for threads on socket 1 reside in socket 0 memory (first-touch allocation by the master thread), causing remote NUMA accesses. A NUMA-aware `first-touch` initialization loop inside the parallel region would mitigate this.

6.9 Q9 – Alternative Approaches

6.9.1 Colouring-Based Domain Decomposition

Partition the mesh into a 2×2 colour pattern such that no two adjacent cells share a colour. Process one colour per phase (four phases total); within each phase all mesh updates are to non-overlapping cells, eliminating all race conditions without extra memory.

- **Advantage:** Zero extra memory (no private grids), zero reduction overhead, fully cache-coherent writes.
- **Disadvantage:** Four sequential phases reduce effective parallelism; load imbalance is likely when particles cluster (one colour's work dominates).

6.9.2 Lock-Free Atomic Accumulation

Use `#pragma omp atomic update` directly on the global mesh. Modern CPUs (AMD Zen 3+, Intel Ice Lake+) implement hardware floating-point atomics with minimal serialisation on lightly-contended cells.

- **Advantage:** No extra memory, no reduction step, simplest code.
- **Disadvantage:** Heavily contended cells (dense particle clusters) cause serialisation and high cache-coherency traffic; typically slower than privatisation for the particle densities in this assignment.

6.9.3 Sorted Particle Approach (Radix Sort)

Sort particles by their (ci, cj) cell index before each scatter. Particles in the same cell are then contiguous; assign each contiguous group to a single thread.

- **Advantage:** No race conditions (each cell processed by exactly one thread), excellent spatial locality, SIMD-friendly.
- **Disadvantage:** Sort cost is $O(N \log N)$ per iteration; for $N = 20\text{ M}$ and 10 iterations this can dominate; requires a parallel sort for efficiency.

6.10 Q10 – Data Structures and OpenMP Strategies for Performance

The following optimisations were applied or explored:

1. **Flat 1D array with row-major layout:** The mesh is stored as a flat `double *` with index `cj * GRID_X + ci`. Compared to a 2D pointer-of-pointers, this eliminates indirection, enables `memcpy/ memset` with SIMD acceleration, and ensures stride-1 reduction loops.
2. **Single contiguous block for all private meshes:** Allocating one block of size $T \times (N_x + 1)(N_y + 1)$ with `calloc` rather than T separate `malloc` calls improves NUMA locality (all allocations from one first-touch region), reduces TLB pressure, and enables efficient cache-line-aligned reduction.
3. **Parallelised reduction with per-slice `omp parallel for`:** The original inner-thread-outer-cell reduction was refactored to outer-thread-inner-cell, converting the

inner loop to a stride-1 sequential read. This alone improved the reduction throughput significantly for C5.

4. **Early-exit for void particles (continue):** Placing the `is_void` check at the very top of the loop body avoids loading particle coordinates for inactive particles, reducing wasted memory bandwidth.
5. **Cell-index clamping:** Boundary checks `if (ci >= NX) ci = NX-1` prevent out-of-bounds writes without branchy modulo operations, keeping the hot path branch-free for the common case.

6.11 Q11 – Interpolation vs. Mover Phase Analysis

Table 9: Phase-level timing breakdown – Cluster, C4 (500×200, 20M particles, 10 iterations)

Phase	T=1 (s)	T=8 (s)	T=16 (s)
Interpolation (scatter)	~6.2	~1.8	~1.1
Normalization	~0.3	~0.05	~0.03
Mover (gather)	~2.1	~0.4	~0.4
Denormalization	~0.3	~0.05	~0.03

The scatter (interpolation) phase is the primary bottleneck. Reasons:

- The scatter loop requires writing to four shared grid locations per particle, requiring either synchronisation (atomics) or the privatisation overhead (extra memory + reduction), whereas the mover only reads from the grid.
- The reduction step (merging T private grids) is an additional serial bottleneck absent in the mover phase.
- The mover is embarrassingly parallel with no synchronisation overhead and scales nearly linearly up to the core count.
- Memory access patterns in the scatter phase are irregular (particle positions determine which cells are written), causing unpredictable cache behaviour. The mover has the same irregular reads, but reads are handled more gracefully by hardware prefetchers than write conflicts.

Recommendation: To improve overall performance, optimising the scatter phase (e.g., via sorting, tiling, or GPU offload) yields the highest return.

7 Conclusion

A correct serial implementation of the full particle-mesh pipeline (scatter → normalize → mover → denormalize) was first validated against the provided test data, then parallelised using OpenMP. The key contributions are:

- **Scatter phase:** Thread-private grids with a cache-friendly parallelised reduction eliminate all data races and scale to the available cores.

- **Mover phase:** Embarrassingly parallel; no synchronisation overhead.
- **Normalization:** OpenMP reduction clauses and parallel loops.

Key findings:

- Maximum speedup of $5.57\times$ achieved for C4 (20 M particles, 500×200 grid) on the 16-core cluster.
- The scatter phase (including reduction) is the dominant bottleneck, responsible for $\sim 70\text{--}80\%$ of per-iteration runtime.
- Cache pressure from large private meshes (C5) limits speedup at high thread counts.
- NUMA effects on the dual-socket cluster cause non-monotone speedup for large-grid configurations at intermediate thread counts.
- Alternative approaches (particle sorting, GPU offload, tiling, MPI+OpenMP) could push performance significantly further.

Acknowledgements

The authors thank Dr. Bhaskar Chaudhury and the teaching assistants (Libin Varghese, Ayushi Sharma, Kaushik Prajapati) for their guidance throughout the assignment.

References

- [1] OpenMP Architecture Review Board, *OpenMP Application Programming Interface, Version 5.2*, 2021.
- [2] G.M. Amdahl, “Validity of the single processor approach to achieving large scale computing capabilities,” *AFIPS Spring Joint Computer Conference*, 1967.
- [3] R.W. Hockney and J.W. Eastwood, *Computer Simulation Using Particles*, Adam Hilger, 1988.
- [4] P.S. Pacheco, *An Introduction to Parallel Programming*, Morgan Kaufmann, 2011.